# Programming Assignment 3 (PA3)

**Magical Demon Hunters Game**
**User-Defined Types, Encapsulation, Operator Overloading, Interacting Objects**
**Out: March 11, 2026 -- DUE: April 7, 2026**
EC327 Introduction to Software Engineering – Fall 2026

**Total: 200 points**

- *You may use any development environment you wish, as long as it is ANSI C++ 11 compatible. Please make sure your code compiles and runs properly under the Linux/Unix environment on the PHO 305/307 (or eng-grid) machines before submitting.*
- *PAs may be submitted up to a week late at the cost of a **30% fixed penalty** (e.g., submitting a day late and a week late is equivalent). It is in your best interest to complete as many checkpoints as possible before the deadline. If you have missing implementations in your original submission, you may complete and submit the missing solutions during the following week. Any submissions after the deadline will be subject to the 30% penalty. No credit will be given to solutions submitted after the 1-week late submission period following the deadline.*
- *Follow the assignment submission guidelines in this document or you will lose points.*

**Submission Format (Must Read)**

- Use the **exact** file names specified in each problem for your solutions.
- Push your changes to the repository generated by GitHub Classroom.
- Complete submissions should have **22 files.**
- Please do **NOT** submit *.exe and *.o or any other files that are not required by the problem.
- **Code must compile in order to be graded. Otherwise, this is an automatic zero.**
- Comment your code (good practice!). We **may** use your comments when grading.

**Coding Style (reminder from PA2)** As you become an experienced programmer, you will start to realize the importance of good programming style. There are many coding "guidelines" out there and it is important that you begin to adopt your own. Naming conventions will help you recognize variable names, functions, constants, classes etc. Similarly, there are many ways to elegantly format your code so that it is easy to read. All of these issues do not affect compilation and hence are easy to overlook at this stage in your development as a programmer. However, your ability (or inability) to create clean, readable code could be the difference in your career when you leave college.

Good reference: https://google.github.io/styleguide/cppguide.html If you find any other good references, feel free to post the link on Piazza!

## Class Specifications

Each class and its members are described below by listing its name, the prototypes for the member functions, and the names and types of the member variables. You **MUST** use the prototypes, types, and names in your program as specified here, in the same upper/lower case. Function argument names are allowed to be different. **Failure to follow these specifications will result in lost points.**

## Final Submission

### View

(15 points)

Thanks to inheritance, we can easily add a better display using simple "ASCII graphics." This display will be like a game board – a grid of squares. Each object will be plotted in the grid corresponding to its position in the plane. As the object moves around its position on the grid will be changed. The object will be identified on the board by its `display_code` letter and its `id_num` value. **We will be assuming that the `id_nums` are all one digit in size at this time.** We will provide sample output to show what the display should look like.

The code for this display will be encapsulated in a class of an object called a "View" whose function is to provide a view of some objects. The `View` object has a member function `Plot()` that puts each object into display grid; it will ask `GameObject` to provide the two characters to identify itself. Member function `Draw()` will output the completed display grid, and `Clear()` will reset the grid to empty in preparation for a new round of plotting.

In class `GameObject`, add the following public member function:

- `void DrawSelf(char * ptr);`
  - The function puts the `display_code` at the character pointed to by ptr, and then the ASCII character for the `id_num` in the next character.

Write the `View` class with its own `.h` and `.cpp` file to have the following members.

#### Constant

defined in the header file

- `const int view_maxsize = 20;`
  - the maximum size of the grid array

**Private members:**

- `int size;`
  - the current size of the grid to be displayed; not all of the grid array will be displayed in this project.
- `double scale;`
  - the distance represented by each cell of the grid
- `Point2D origin;`
  - the coordinates of the lower left-hand corner of the grid
- `char grid[view_maxsize][view_maxsize][2];`
  - an array to hold the characters that make up the display grid.
- `bool GetSubscripts(int &out_x, int &out_y, Point2D location);`
  - This function calculates the column and row subscripts of the grid array that correspond to the supplied location. Note that the x and y values corresponding to the subscripts can be calculated by (location - origin) / scale. Assign these to integers to truncate the fractional part to get the integer subscripts, which are returned in the two reference parameters. The function returns `true` if the subscripts are valid, that is within the size of the grid being displayed. If not, the function prints a message: **"An object is outside the display"** and returns `false`.

**Public members:**

- `View()`
  - It sets the size to 11, the scale to 2, and lets the origin default to (0, 0). No constructor output message is needed.
- `void Clear();`
  - It sets all the cells of the grid to the background pattern shown in the sample output.
- `void Plot(GameObject * ptr);`
  - It plots the pointed-to object in the proper cell of the grid. It calls `get_subscripts` and if the subscripts are valid, it calls `DrawSelf()` for the object to insert the appropriate characters in the cell of the grid. However, if there is already an object plotted in that cell of the grid, the characters are replaced with '*' and ' ' (space) to indicate that there is more than one object in that cell of the grid.

**Tip about base class pointers:** C++ normally refuses to convert one pointer type to another. But it will allow a conversion from a Derived class pointer to a base class pointer (upcasting). This allows you to plot all kinds of `GameObjects` with a single function.

**Tip about C/C++ arrays:** If `a` is a three-dimensional array, then `a[i][j][k]` is the i, j, k element in the array, and `a[i][j]` is a pointer to a one-dimensional array starting `a[i][j][0]`

- `void Draw();`
  - Outputs the grid array to produce a display like that shown in the sample output. The size, scale, and origin are printed first, then each row and column, for the current size of the display. Note that the grid is plotted like a normal graph: larger x values are to the right, and larger y values are at the top. The x and y axes are labeled with values for every alternate column and row. Use the output stream formatting facilities to save the format settings, set them for neat output of the axis labels on the grid, and then restore them to their original settings.
  - Specifications: Allow two characters for each numeric value of the axis labels, with no decimal points. The axis labels will be out of alignment and rounded off if their values cannot be represented well in two characters. This distortion is acceptable in the name of simplicity for this project

Now modify your main program to declare a view object. Where you previously did a `ShowStatus()` call for each object, replace that code with a first call to the `View` object's `Clear()` function, then call the plot function using each object, and then the `Draw()` function. Now you should have the graphical display. Thanks to the inheritance from `GameObject`, the display works for all three types of specific game objects.

## Overall Structure of the Program (main.cpp)

(35 points for behavioral tests; an update will be posted on Blackboard regarding this requirement and sample output)

The main program should include a loop that reads a command, executes it, and then asks for another command. **You do NOT have to detect errors or bad input. PA4 will add these features.**

The program will declare two `Mage` objects, two `ManaSpire` objects, and two `DemonHideout` objects as outlined in the `Model` class description. These objects will persist throughout the execution of the program.

The program starts by displaying the current time and the status of each object using the show status function. The `main` function then asks for a command and the user inputs a single character for the command, and main calls a function for the appropriate command. The function for the command inputs requests any required additional information from the user, and then carries out the command. If the user entered the "advance" or "run" commands, the program repeats the main loop by displaying the time and current status and prompting for a new command. Otherwise, it continues to prompt for new commands. This enables the user to command more than one object to move before starting the simulation running again.

## Additional Specifications

You MUST:

- Make use of the classes and their members; you may not write non-object oriented code to do things that the classes can do.
- Declare function prototypes for the functions that are not part of a class, such as those called by `main()`, and list the function definitions after `main`. This will improve the readability of your program.
- Make `.h` and `.cpp` files for each of your classes with data fields and member functions as specified above.
- Make sure your code compiles!

### Programming Guideline

Unless you have so much experience that you shouldn't be in this course, trying to write this program all at once is the *hard* way to do it! Object oriented programming is easy to do in chunks! That's the idea! The individual classes can be written and tested pretty much one at a time, and a piece at a time. Here's how to write this program a chunk at a time:

1. Start by writing the `Point2D` and `Vector2D` classes and a couple of their functions. Write a trivial version of `main` to test them by creating one of each and doing things with them. Add the additional functions one at a time, and test each one. Getting the overloaded output operator to work soon will make testing the remaining functions more fun. Also, write the `GameObject` class and test with your trivial `main`.

2. Write the `ManaSpire` and `DemonHideout` classes. Add to your testing `main` function a declaration of an `ManaSpire` and `DemonHideout` object, and call `DemonBattle()` and `DistributeCrystal()`, then `Update()` and `ShowStatus()` on the objects multiple times

3. Start on writing the `Mage` class, but only write the constructors and a partial form of the `ShowStatus` function; leave everything else out. Write another trivial testing `main`; create a `Mage` object and a `ShowStatus()` function. It should display the correct initial state of the object to verify both the constructors and the `ShowStatus` function. Then, add the `StartMoving()`, `Stop()`, `SetupDestination()`, and `UpdateLocation()` functions. Write the `ShowStatus()` and `Update()` functions. After testing this class, follow the same procedure to make the `Mage` class. When working on `ShowStatus()` and `Update()`, add the `STOPPED` and `MOVING` parts first. Change your trivial main function to call the `StartMoving()` function on your one `Mage` object. This should show its status, call its `Update()` function, and show its status again - it should be in a different location! Check that the amount moved on the step is correct. Put in a few more calls of update and see if the `Mage` stops as it should.
4. Write the command loop for `main()`, and put the whole program together, and test it by making the objects all move around. See if you can have the two `Mages` recover in two `ManaSpires` in various combinations at the same time.

### Makefile

Makefiles automate and facilitate the compilation process of software projects that consists of a large number of source code (`.cpp`) files (such as PA3). Instead of specifying each `.cpp` file in the compilation process (g++) of PA3, we will create makefiles to automate the compilation process and the achievement of the checkpoints.

In general, a makefile consists of multiple named targets ("rules") that can depend on each other. Every target has an associated action that is usually a UNIX command, such as g++. The structure of a named target looks as follows:

```
target_name : dependencies
    command_to_execute
```

**IMPORTANT!** The `command_to_execute` line MUST be indented using a Tab space. This can be achieved by using the Tab key on your keyboard.

The UNIX `make` command interprets makefiles by executing the targets and their associated actions in the order of the targets' dependencies. Per default, the make command executes a makefile that is literally called `Makefile` (case-sensitive!). In case of naming your Makefile differently (e.g. `Makefile_Checkpoint1`), then you must use the `-f` option for the make command. Also check out the manual page of the make command (`man make`).
To demonstrate makefiles, we provide a makefile for Checkpoint I. Let's name the makefile `Makefile_Checkpoint1`.

```
1  # in EC327, we use the g++ compiler
2  # therefore, we define the GCC variable
3  GCC = g++
4
5  # a target to compile the Checkpoint1 which depends on all object-files
6  # and which links all object-files into an executable
7  Checkpoint1: TestCheckpoint1.o Point2D.o Vector2D.o
8      $(GCC) TestCheckpoint1.o Point2D.o Vector2D.o -o Checkpoint1
9
10 # a target to compile the TestCheckpoint1.cpp into an object-file
11 TestCheckpoint1.o: TestCheckpoint1.cpp
12     $(GCC) -c TestCheckpoint1.cpp
13
14 # a target to compile the Point2D.cpp into an object-file
15 Point2D.o: Point2D.cpp
16     $(GCC) -c Point2D.cpp
17
18 # a target to compile the Vector2D.cpp into an object-file
19 Vector2D.o: Vector2D.cpp
20     $(GCC) -c Vector2D.cpp
21
22 # a target to delete all object-files and executables
23 clean:
24     rm TestCheckpoint1.o Point2D.o Vector2D.o Checkpoint1
```

As demonstrated, makefiles only support single-line comments that start with the '#' character. Furthermore, makefiles support the definition of variables, such as GCC, which get a specific value assigned. In our case, the GCC variable holds the value g++. You can access the value of the variables using the `$()` notation. For example, `$(GCC)` returns the value of the GCC variable, which is g++.

In the file `Makefile_Checkpoint1` we specify five targets (the name before the colon on lines 7, 11, 15, 19, and 23). The `Checkpoint1` target depends on three targets, namely `TestCheckpoint1.o`, `Point2D.o`, and `Vector2D.o`. Each `*.o` target depends on the `.cpp` file, which should be compiled to an object file (using the `-c` option of the g++ compiler). We define those dependencies in order to avoid re-compilation of `.cpp` files that have not been changed during the last compilation process. We also define one target clean, which deletes all object-files and the executable `Checkpoint1` (using the `rm` UNIX command).

Per default, the make command executes the first target, which is in our case `Checkpoint1`. You can, however, instruct what target to execute by passing the name of the target to the make command. For example, the following command will execute the clean target of the `Makefile_Checkpoint1` makefile.

```
1  make -f Makefile_Checkpoint1 clean
```

The EC327 staff will provide a final makefile for PA3. However, it is your task to formulate the makefiles

for each checkpoint.

For further information on makefiles, please consult https://www.gnu.org/software/make/

**Submission**
Clone the PA3 branch from your class GitHub repository. Write all of your code in the cloned directory. Please use the file names **Point2D.h, Point2D.cpp, GameObject.h, GameObject.cpp, Vector2D.h, Vector2D.cpp, Building.h, Building.cpp, ManaSpire.h, ManaSpire.cpp, DemonHideout.h, DemonHideout.cpp, Mage.h, Mage.cpp, Model.h, Model.cpp, View.h, View.cpp, GameCommand.h, GameCommand.cpp, and main.cpp**. After each checkpoint or major change, make commits to your local repository. When you are ready to submit, push your changes to the repository. Do **NOT** commit or submit your executable files (a.out or others) or any other files in the folder, it will be impractical for git to handle. Make sure to add **comments** to your code.